Lab Assignment 5

Title: Design and implement a hash table of fixed size. Use the division method for the hash function and resolve collisions using linear probing. Allow the user to perform the following operations:
• Insert a key
• Search for a key
• Delete a key
• Display the table

Program code:

```
#include <iostream>
#include <iomanip>
using namespace std;

const int TABLE_SIZE = 11;   // keep this prime so probing works well
const int R = 7;             // prime smaller than TABLE_SIZE (for double hashing)

enum State { EMPTY, OCCUPIED, DELETED };

struct Slot {
    int key;
    State state;
};

Slot table[TABLE_SIZE];

// ---------- Hash functions (division method) ----------
int hash1(int key) {
    return key % TABLE_SIZE;
}

int hash2(int key) {           // secondary hash for double hashing (never 0)
    return R - (key % R);
}

// ---------- Probe functions: i-th probe position for a key ----------
int linearProbe(int key, int i) {
    return (hash1(key) + i) % TABLE_SIZE;
}

int quadraticProbe(int key, int i) {
    return (hash1(key) + i * i) % TABLE_SIZE;
```

```
}

int doubleHashProbe(int key, int i) {
    return (hash1(key) + i * hash2(key)) % TABLE_SIZE;
}

// Pointer type so one set of operations can use any probe function
typedef int (*ProbeFn)(int, int);

// ---------- Generic operations ----------
void initTable() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        table[i].key = -1;
        table[i].state = EMPTY;
    }
}

int searchKey(int key, ProbeFn probe) {
    for (int i = 0; i < TABLE_SIZE; i++) {
        int idx = probe(key, i);
        if (table[idx].state == EMPTY) return -1;            // never used: key absent
        if (table[idx].state == OCCUPIED && table[idx].key == key) return idx;
    }
    return -1;
}

bool insertKey(int key, ProbeFn probe) {
    if (searchKey(key, probe) != -1) {
        cout << "Key " << key << " already exists.\n";
        return false;
    }
    for (int i = 0; i < TABLE_SIZE; i++) {
        int idx = probe(key, i);
        if (table[idx].state != OCCUPIED) {                  // EMPTY or DELETED
            table[idx].key = key;
            table[idx].state = OCCUPIED;
            cout << "Inserted " << key << " at index " << idx
                 << " (probes: " << i + 1 << ")\n";
            return true;
        }
    }
    cout << "Could not insert " << key << " (table full or no free slot on probe path).\n";
    return false;
}
```

```
bool deleteKey(int key, ProbeFn probe) {
    int idx = searchKey(key, probe);
    if (idx == -1) {
        cout << "Key " << key << " not found.\n";
        return false;
    }
    table[idx].state = DELETED;                          // tombstone keeps probe chains intact
    cout << "Deleted " << key << " from index " << idx << "\n";
    return true;
}

void displayTable() {
    cout << "\nIndex | Value\n--------------\n";
    for (int i = 0; i < TABLE_SIZE; i++) {
        cout << setw(5) << i << " | ";
        if (table[i].state == OCCUPIED)      cout << table[i].key;
        else if (table[i].state == DELETED)  cout << "<deleted>";
        else                                 cout << "-";
        cout << "\n";
    }
    cout << endl;
}

// ---------- Separate functions per technique ----------
void linearProbing(int op, int key) {
    switch (op) {
        case 1: insertKey(key, linearProbe); break;
        case 2: {
            int idx = searchKey(key, linearProbe);
            if (idx != -1) cout << "Found " << key << " at index " << idx << "\n";
            else cout << "Key " << key << " not found.\n";
            break;
        }
        case 3: deleteKey(key, linearProbe); break;
    }
}

void quadraticProbing(int op, int key) {
    switch (op) {
        case 1: insertKey(key, quadraticProbe); break;
        case 2: {
            int idx = searchKey(key, quadraticProbe);
            if (idx != -1) cout << "Found " << key << " at index " << idx << "\n";
```

```
            else cout << "Key " << key << " not found.\n";
            break;
        }
        case 3: deleteKey(key, quadraticProbe); break;
    }
}

void doubleHashing(int op, int key) {
    switch (op) {
        case 1: insertKey(key, doubleHashProbe); break;
        case 2: {
            int idx = searchKey(key, doubleHashProbe);
            if (idx != -1) cout << "Found " << key << " at index " << idx << "\n";
            else cout << "Key " << key << " not found.\n";
            break;
        }
        case 3: deleteKey(key, doubleHashProbe); break;
    }
}

// ---------- Main ----------
int main() {
    int method = 0;
    cout << "Hash Table (size " << TABLE_SIZE << ", division method h(k) = k % " <<
TABLE_SIZE << ")\n";
    cout << "Choose collision resolution:\n"
         << "  1. Linear Probing\n"
         << "  2. Quadratic Probing\n"
         << "  3. Double Hashing\n"
         << "Enter choice: ";
    cin >> method;
    if (method < 1 || method > 3) {
        cout << "Invalid choice.\n";
        return 1;
    }

    initTable();

    int choice, key;
    do {
        cout << "\n--- MENU ---\n"
             << "1. Insert a key\n"
             << "2. Search for a key\n"
             << "3. Delete a key\n"
```

```
            << "4. Display the table\n"
            << "5. Exit\n"
            << "Enter choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 3) {
            cout << "Enter key: ";
            cin >> key;
            if (method == 1)      linearProbing(choice, key);
            else if (method == 2) quadraticProbing(choice, key);
            else                  doubleHashing(choice, key);
        } else if (choice == 4) {
            displayTable();
        } else if (choice != 5) {
            cout << "Invalid choice.\n";
        }
    } while (choice != 5);

    return 0;
}
```

Output:

```
Hash Table (size 11, division method h(k) = k % 11)
Choose collision resolution:
  1. Linear Probing
  2. Quadratic Probing
  3. Double Hashing
Enter choice: 1

--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 1
Enter key: 22
Inserted 22 at index 0 (probes: 1)

--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
```

5. Exit
Enter choice: 1
Enter key: 33
Inserted 33 at index 1 (probes: 2)

--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 1
Enter key: 44
Inserted 44 at index 2 (probes: 3)

--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 4

Index | Value
----
    0 | 22
    1 | 33
    2 | 44
    3 | -
    4 | -
    5 | -
    6 | -
    7 | -
    8 | -
    9 | -
   10 | -

--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 2
Enter key: 33
Found 33 at index 1

```
--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 3
Enter key: 33
Deleted 33 from index 1

--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 4

Index | Value
--------------
    0 | 22
    1 | <deleted>
    2 | 44
    3 | -
    4 | -
    5 | -
    6 | -
    7 | -
    8 | -
    9 | -
   10 | -


--- MENU ---
1. Insert a key
2. Search for a key
3. Delete a key
4. Display the table
5. Exit
Enter choice: 5
```